EchoPin: 30-minute live demo script

About 27 minutes of talk and demo, with 3 minutes spare. If questions have to fit inside the 30 minutes, drop the sections marked cut if short. That frees about 5 minutes.

How to read this. Plain paragraphs are what you say. Lines marked ▶ are what you do on screen. Each heading gives the clock time you should have reached when that section ends.

Every on-screen message quoted below was checked against the code in v0.3.0. The four invite-code errors in Part 2 were run through the real parser.

Part 0: Opening ends 2:30

> ▶ Home screen on the projector. Phone on silent.

Hands up if you've ever lost someone in a crowd. At a festival, in a mall, on a school trip, or on a hike where someone fell behind.

(pause, let hands go up)

What did you do? You probably phoned them, and they didn't hear it. Or you opened a location-sharing app.

Location sharing works, but look at what it costs. To show you where your friend is, their phone sends their exact GPS coordinates to a company's server all day. That company now knows where they sleep, where they work, and who they spend time with. And on a mountain or in a packed stadium with no signal, the app stops working anyway, because it needs that server.

So I asked a narrower question. When you're out with a group, what do you actually need to know? You don't need a map of the city. You need three things. Is everyone still close? Is everyone OK? And if someone is in trouble, can they tell all of us at once?

EchoPin answers those three questions without a server, without an account, and without a GPS coordinate ever existing. For the next half hour I'll show it working live. Then I'll open it up and show you what's underneath, including the parts that aren't finished yet.

Part 1: What EchoPin is ends 5:00

EchoPin makes three promises.

First: relative, not absolute. EchoPin never knows where you are. It only knows roughly how far away your friends are, and it gets that from the strength of their Bluetooth signal, measured on your own phone. There is no latitude or longitude field anywhere in the code. I'm not saying "we encrypt your location." I'm saying your location doesn't exist in this system.

Second: no servers. Phones talk to each other directly. They use Bluetooth Low Energy when there's nothing else, and Wi-Fi or a phone hotspot when you need more range. Both run at the same time, and if a message arrives over both, the copy is dropped.

Third: the code is the key. A group starts a session and shares an invite code. Every message is encrypted and signed with keys that come from that code. Without the code, nobody can read the session. That includes me, the developer.

It's built in Flutter, so one codebase runs on Android, iOS, macOS, Linux, Windows, and the web. The web version is demo only, because browsers can't use these radios. On Android and iOS, the native code only moves bytes. All the cryptography is in Dart, so it's identical on every platform and I can test it on a laptop.

Part 2: Home screen and invite codes ends 8:00

> ▶ Point at the top of the home screen.

This is the whole home screen. Your name, *Start a session*, *Join a session*, and a demo button. At the bottom it reminds you: "No coordinates ever leave this device."

Look at the hint in the invite box: ECHO, sixteen characters, then two more. The sixteen characters are the session secret, about 78 bits of randomness. The last two are a checksum, so a typo gets caught before you try to join.

The alphabet has 30 characters. There's no zero or letter O, no one, I or L, and no U, because those are the ones people misread when someone reads a code out loud across a campsite.

Let me get it wrong on purpose.

> ▶ Type `ECHO-1234` in **Invite code**, press **Join session**. Screen shows: *"Codes never contain 1. Check for 0/O, 1/I/L or U"*

It doesn't just say "invalid." It tells me which character is wrong and what I probably meant.

> ▶ Clear the field. Type `ECHO-ABCD-EFGH`, press **Join session**. Screen shows: *"Codes have 16 characters after ECHO, plus a 2-character check (got 8)"*

> ▶ Paste `ECHO-PARK-WAYS-7JG3-MZQ2-RJ` (keep it in a note before you start), press **Join session**. Screen shows: *"That code does not check out. Look for a typo"*

That one is the right length and uses only valid characters, but I changed one letter from a real code. The checksum caught it. The check needs no secret, so it works on any phone. In an earlier version it only worked on the phone that created the code, and fixing that was part of this release.

You can also join by scanning a QR code or tapping an `echopin://join` link. All three paths go through the same parser, so they all give the same precise errors.

This is a rule across the whole app: every failure is a typed error with a sentence a person can act on. You'll see it again when we get to Bluetooth.

Part 3: The pinwheel ends 13:00

I can't count on four volunteers' phones in a room like this, so EchoPin has a demo mode. Here is exactly what it is. The four people are simulated, but each one is a complete copy of the session engine, with its own cryptographic identity, joined to the session through an in-memory radio. Every message you're about to see is really encrypted, signed, and checked. Only the radio and the people are pretend.

> ▶ Tap **Try the demo (simulated people)**. "Starting the simulation…" appears, then Amara, Ben, Chloe and Dev.

This is the pinwheel. You're in the middle. Each dot is a person, and the distance from the centre is how far away they are. The dashed circle is the boundary, 150 metres by default.

Watch Ben. He starts at about 142 metres, right on the line, and in demo mode people wander, so he'll probably cross it while I'm talking.

One honest detail: Bluetooth tells you *how far*, not *which way*. So the angle of each dot in a real session is a fixed position generated from the person's ID. It keeps them in the same place so you can find them again, but it isn't a compass bearing. The distance is the real information.

> ▶ Drag on the pinwheel to shrink the boundary to about 80 m. (The **Boundary** slider does the same.)

When someone crosses the boundary you get an alert. On a phone that's a banner, a vibration, and a system notification.

> ▶ Point at the banner, e.g. *"Ben left the boundary (…m)"*.
> ▶ Drag the boundary back out to about 200 m.

Notice which distance triggers the alert: the *blurred* one. Every distance is deliberately blurred by up to plus or minus 7 metres. The blur is fixed per person for a minute, so the dots don't jitter. It's added before the boundary check and before anything reaches the screen, so even the app only ever works with the blurred number.

> ▶ Point at the people list (a side panel on a laptop, a sheet you drag up on a phone).

This is the people tray. A card slides in when someone appears and out when they leave. Each card shows distance, which radio we heard them on (here it says *Demo*, in real use *Bluetooth* or *Wi-Fi*), and the status of any ping.

In a real session, there's a case you won't see in the demo. If a phone is advertising your session over Bluetooth but hasn't yet sent a signed message, it's drawn hollow and labelled *"Unconfirmed signal."* It can't trigger alerts and you can't ping it. Anyone can broadcast a fake Bluetooth advertisement. Nobody can fake a signature.

People reached only over Wi-Fi appear in the list but not on the pinwheel, because Wi-Fi gives no distance, and EchoPin won't draw a dot it can't justify.

> ▶ Tap **Amara**. Her sheet shows *"About … away"* and two safety codes.

Every member gets a fresh 12-digit safety code in every session. If Amara's phone shows the same number for herself that mine shows for her, then the messages I'm getting really come from her phone and not from an impostor. It's the same idea as safety numbers in Signal or WhatsApp.

> ▶ Tap **Codes match**. A **Verified** badge appears.

Part 4: Check-ins and SOS ends 17:30

> ▶ Close Amara's sheet. Tap **Check in with everyone**.

There are only three answers: *I'm OK*, *On my way*, and *Need help*. That's deliberate. If you're lost and stressed, you shouldn't have to type.

> ▶ Point at the outgoing tray as each person moves from *Pinging…* to *Delivered* to an answer (usually *I'm OK*, sometimes *On the way*).

Each status has a precise meaning. *Delivered* means their phone sent back a signed receipt. Then their answer arrives. If a phone doesn't send a receipt, mine resends every 4 seconds. A check-in gives up after 8 tries and marks that person *No answer*. Bluetooth drops packets, so I'd rather retry than pretend a message arrived.

Now say I only want to check on two people.

> ▶ Long-press **Ben**, then tap **Chloe**. The header reads "2 selected" and the button changes to **Check in with 2**. Tap it. If it's been under 10 seconds since the last check-in, you'll see *"Slow down. Try again in … s"*.

(If the "Slow down" message appears:) That's on purpose. Each phone can send one check-in every 10 seconds, so one nervous person can't flood everyone's radio and battery. *(Wait, then tap again.)*

> ▶ Tap **Clear** on the finished check-in trays.

Now the important one.

> ▶ Tap **SOS** once, quickly. Screen shows: *"Press and hold SOS to alert everyone in the session"*

A quick tap only explains itself, because a press in your pocket shouldn't start an emergency.

> ▶ Press and hold **SOS** (about one second) until it fires. Screen shows: *"SOS sent to everyone. Keep EchoPin open."* The tray reads **Your SOS is live**, and within a few seconds the demo people answer *On my way*.

An SOS differs from a check-in in three ways. It keeps retrying much longer: 30 attempts instead of 8. It stays live, so anyone who joins the session while it's active receives it automatically. And you can send it when nobody is in range yet, and it goes out the moment someone appears.

On the receiving phone, an SOS always raises a notification, even if notifications are switched off, plus three strong vibrations. The person in trouble also gets a pulsing red halo on everyone's pinwheel.

> ▶ Tap **I'm safe**.

One tap resolves it on every phone.

In the demo the simulated people only answer pings. They never start them. So what you've seen is the sender's side. On a receiving phone, a check-in shows up as a card saying *"Amara is checking in"* with the three answer buttons.

Part 5: When things go wrong ends 19:30 cut if short

Outdoors, the most common failure isn't a hacker. It's Bluetooth being switched off.

When something is wrong, a status tray slides in at the top, says exactly what's wrong, and fixes it in one tap where it can:

- **Bluetooth is off**, with a **Turn on** button that brings up the system prompt.
- **Bluetooth not allowed**, with **Allow**, or **Settings** if it's been permanently blocked.
- **Not on Wi-Fi**: join the same network or hotspot as your group.
- **You can't reach anyone**: this one can't be dismissed, and pings are held and sent the moment a radio comes back.
- **Security warning**: someone tried to impersonate a member.

> ▶ *(Only with an Android phone in a real session.)* Swipe down and switch Bluetooth off. The **Bluetooth is off** tray slides in. Tap **Turn on**.

And one rule I won't break: a real session never falls back to fake people. If no radio works, you get an honest message, not a comforting picture.

Sessions also end by themselves after 8 hours. When you leave, your phone sends a signed goodbye and wipes every key.

Part 6: Under the hood ends 24:00

> ▶ Optional: open `docs/PROTOCOL.md` on screen, or show it as a slide.

Here's what happens to that invite code.

From code to keys. The 16-character secret goes through HKDF, a standard key-derivation function, which produces two values. One is an 8-character public session ID that phones advertise so they can find each other. The other is a 256-bit encryption key that never leaves the phone. HKDF only works one way, so seeing the session ID tells you nothing about the code.

Identity. Each phone creates a fresh Ed25519 signing key for every session. Not once per install: once per session. If you're in a session today and another tomorrow, nobody can link the two.

The envelope. Messages are encrypted with XChaCha20-Poly1305, and the message *type* is inside the encryption too. Someone sniffing Bluetooth can't tell a "hello" from an SOS.

Seven checks. Every incoming message passes seven checks, in order, and is dropped at the first one it fails:

1. Right format, version, and session. Traffic from other sessions is ignored silently.
2. It decrypts, which proves the sender holds the code.
3. The signature is valid.

4. The sender's ID matches their key, and the ID is pinned to that key the first time we see it. A different key claiming a known ID is an impersonation attempt and raises the security warning.
5. A replay window of 64 messages per sender. This is also where the same message arriving over both Bluetooth and Wi-Fi gets dropped, so running two radios costs nothing extra.
6. The timestamp is within two minutes of our clock.
7. A rate limit of five messages in a burst, refilling one every half second, and a cap of 20 members per session.

Names and notes from the network count as hostile input. They're stripped of control characters, including the right-to-left override trick that makes text display backwards to disguise it, and capped at 24 and 80 characters.

A design change I want to be upfront about. The original architecture planned a separate Noise handshake between every pair of phones. When I built SOS, I realised EchoPin's traffic is group broadcast: one SOS goes to everybody. Pairwise handshakes for 20 people is 190 handshakes over a slow radio. So version 2 uses one group key from the code, plus a signature from every device. The trade-off is forward secrecy. If someone gets the code later, they could decrypt traffic they recorded from that session. I accepted that because sessions last at most 8 hours and keys are never reused, and I wrote it into the protocol document rather than hiding it.

Architecture, in one breath. Kotlin and Swift plugins advertise, scan, and move opaque bytes. Everything else is Dart: radios feed a secure messenger, which feeds the session engine, which feeds the screen. App state lives in four of Flutter's own InheritedNotifier scopes, with no third-party state-management library.

At rest. Saved invite codes for *Recent sessions* go in the phone's keystore, wherever the platform has one. People, distances, and pings are never written to disk.

Part 7: The admin system ends 26:00 cut if short

> ▶ Open the ⋮ menu → **Admin**, and enter your PIN.

There's a PIN-protected admin panel for diagnostics and testing. The PIN is stored only as a salted PBKDF2 hash. Five wrong tries lock it for 60 seconds, and the lockout survives restarting the app. It locks itself after 5 minutes idle. Every privileged action re-checks the lock in the logic layer, not just on the screen, so getting to the screen some other way doesn't get you in.

> ▶ Scroll through **Diagnostics**: messages sent, accepted and rejected, your peer ID and safety code.

Notice what isn't here: the invite code. Not even the admin can see it.

> ▶ In **Synthetic peers**, label one "Test", set **Distance** to 100 m, tap **Inject**. Go back to the session: Test appears inside the boundary.
>
> ▶ Return to Admin, set **Distance** to 300 m, tap **Move to slider position**, and go back. The banner reads *"Test left the boundary …"*.

This is how I test boundary alerts without walking around a field. Synthetic peers go through exactly the same blur and crossing logic as real ones.

> ▶ Glance at **Audit log (this run only)**, then lock the panel.

Part 8: What's proven and what isn't ends 28:00

Here's what's verified. There are 159 automated tests, and they all pass. They cover:

- Security: tampering, wrong keys, impersonation, replay, clock skew, flooding, and the member cap.
- Whole sessions in memory, including lossy links, network splits, and late joiners.
- Two complete copies of the app on a shared radio, exchanging pings, SOS, and *I'm safe*.
- The UI end to end, including the hold-to-send SOS.

On top of that, an integration test runs real Bonjour discovery and real TCP between two engines on Linux. Continuous integration builds for Android, web, Linux, iOS, macOS, and Windows.

Here's what isn't verified yet:

- **Bluetooth between two real phones.** The Android and iOS Bluetooth code compiles, and the transport is tested against a simulated radio, but it hasn't met real hardware. That's the very next step: Android to Android, then Android to iPhone.
- **iOS and macOS builds** need a Mac. CI runs them on macOS machines.
- **Running in the background.** Bluetooth discovery works while the app is open. iOS doesn't advertise names in the background, and Android may pause scanning with the screen off.

After that comes a background service, field trials outdoors to measure real distance accuracy and battery use, usability testing, and an outside security review.

Part 9: Close ends 29:30

Back to the festival. With EchoPin, your group shares one code. Everyone can see roughly how far everyone else is. If someone drifts past the boundary, you know. One tap checks on everyone, and one long press tells the whole group you need help. No signal needed, no server, no coordinates, and nobody outside the group can read a word of it.

Location apps answer the question "where is everyone?" EchoPin answers a smaller one: "is everyone OK?" I think the smaller question is the one that matters.

Thank you. I'm happy to take questions.

Appendix A: Setup checklist

The day before

- [ ] Run the app you'll present (`flutter run -d linux --release`, or the Android APK mirrored to the screen).
- [ ] Set your display name on the home screen.
- [ ] Create the admin PIN in advance and write it down. There's no remote reset, and a forgotten PIN means clearing app data.
- [ ] Do one full run of this script with a timer.
- [ ] Put the typo code `ECHO-PARK-WAYS-7JG3-MZQ2-RJ` in a note you can copy from.

Fifteen minutes before

- [ ] Phone and laptop on Do Not Disturb, other apps closed.
- [ ] Start the app fresh, on the home screen, with the invite field empty.
- [ ] Make the window big enough that the pinwheel and people panel are readable from the back row.
- [ ] Have a terminal open in the repo, for Appendix B.

Appendix B: If something breaks

Problem	What to do and say
The demo won't start	Restart the app. "Live demos: this is why I wrote 159 tests."
No boundary alerts appear	Drag the boundary smaller, below Ben and Dev. Alerts are limited to one every 5 s per person.
No answers to a check-in	Wait 5 seconds (demo replies take 1.5–4.5 s), then say: "It retries every 4 seconds until it gets a receipt."
Admin won't unlock	Skip Part 7 and talk through the bullets.
The app is unusable	Switch to the terminal and run `flutter test test/security/secure_messenger_test.dart`. 20 security tests (tampering, replay, impersonation, flooding) pass in a few seconds. Walk through the test names while they run.

Appendix C: Likely questions

How accurate is the Bluetooth distance? Rough. Signal strength changes with walls, bodies, and how you hold the phone. That's why the app says "about 40 m" and blurs it on top of that. Measuring accuracy outdoors is on the roadmap.

What's the range? It depends heavily on the phones and the surroundings, and I haven't measured it on real devices yet. Wi-Fi or a hotspot extends it beyond Bluetooth.

What if someone shares the code with an outsider? Then that person is a member. Holding the code *is* membership, by design. It's documented as out of scope. Sessions cap at 20 people and end after 8 hours, and you can always start a new one.

Why not just use GPS? Coordinates are the privacy problem. Version 0.1 actually used GPS with mathematical noise added. Version 0.3 removed GPS entirely, because the safest coordinate is one that's never measured.

Can you, the developer, read sessions? No. There's no server to read from, and the key comes from a code that only passes between the people in the group.

Did you invent your own cryptography? No. HKDF-SHA256, XChaCha20-Poly1305, and Ed25519 are all standard, from the `cryptography` package. How they're combined is specific to EchoPin, and that combination needs an outside review. It's on the roadmap.

Why no direction arrow? Bluetooth only gives signal strength, not angle. Ultra-wideband radios on newer phones can measure direction, and the original roadmap lists that as a possible future addition for phones that support it.

What happens when person 21 tries to join? They're refused, and everyone sees a "Session is full" tray.

Battery life? Not measured yet. It's part of the field trials.

Why Flutter? One codebase for six platforms, and the cryptography is identical everywhere, so testing it on a laptop proves it for the phones.

Appendix D: Numbers to remember

Invite code	`ECHO-` + 16 characters + 2 check characters, about 78 bits, 30-character alphabet
Default boundary	150 m (adjustable 50–500 m)
Privacy blur	±7 m, fixed per person per minute
Members per session	20
Session lifetime	8 hours

Safety code	12 digits, new every session
Retries	every 4 s; check-in 8 tries, SOS 30
Check-in limit	one per 10 s per phone
SOS	hold about 1.2 s
Replay window	64 messages; clock tolerance ±2 min
Rate limit	5-message burst, refill 1 per 500 ms
Admin PIN	4–12 digits, PBKDF2; 5 tries, then a 60 s lockout; auto-lock after 5 min
Tests	159 unit and widget tests passing, plus a real-network integration test

Appendix E (optional): a real session over Wi-Fi

Use this only if you have rehearsed it at the venue. A Linux laptop talking to an Android phone over Wi-Fi hasn't been tested yet. The integration test runs two engines on one Linux machine. If it works in rehearsal, it's the strongest moment in the talk, because the audience sees real encrypted traffic and the receiving side of a check-in. It fits after Part 4 and takes about 3 minutes.

1. Put both devices on the same network. A pocket router or a third phone's hotspot is safest, because venue Wi-Fi often blocks traffic between devices. On Linux, `avahi-daemon` must be running.
2. Laptop: **Create session** with a name, then tap **Invite** to show the QR code.
3. Phone: tap the scan icon in **Invite code** and scan the QR code.
4. Laptop: the phone appears in the people tray as *Wi-Fi · Distance unknown*, listed but not on the pinwheel.
5. Laptop: **Check in with everyone**. Phone: the check-in card appears. Tap **I'm OK**. Laptop: the answer arrives.

What to say: "That was a real session. Encrypted, signed, and verified on both ends, with no server between us. The phone is in the list but not on the pinwheel, because over Wi-Fi there's no signal strength to measure, and EchoPin won't make up a distance."